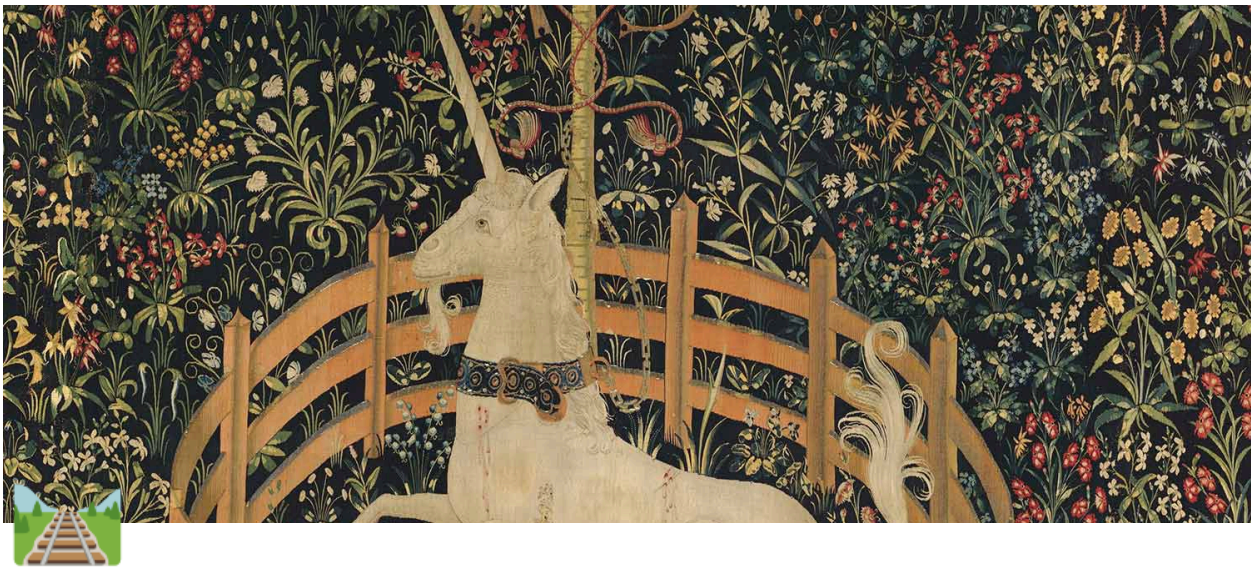

Shannon Agentic AI Foundation

▼ OVERVIEW

AI Software Factory Blueprint

Agentic AI Engineering Organization

Version 1.0

---

1. Overview

This document describes the **end-to-end architecture** for building an **Agentic AI Software Factory**.

The system allows AI agents to collaborate as a **software engineering organization**, capable of:

- designing software systems
- generating implementation code

- reviewing quality
- learning from experience
- improving over time

The architecture is divided into **three major layers**:

1. **AI Foundation Layer** (how the AI thinks and behaves)
2. **AI Orchestration Layer** (how agents run and collaborate)
3. **Software Execution Layer** (how software is generated and deployed)

---

2. System Vision

The goal is to create a system where **multiple specialized AI agents ("Chappies") collaborate like a real engineering team**.

Example team:

Agent	Role
Product Agent	Defines product requirements
Architect Agent	Designs system architecture
Engineer Agent	Implements code
QA Agent	Reviews correctness
DevOps Agent	Prepares deployment

These agents operate under **shared rules and philosophy**, defined by the **AI Foundation**.

---

3. High-Level Architecture

```
Human
   ↓
AI Orchestrator (OpenClow / Orchestrator)
   ↓
AI Agent Team
```

```
   ↓
Project Repository
   ↓
Generated Software
```

The orchestrator manages the AI agents and ensures they follow the foundation rules.

---

4. AI Foundation Layer

Before any agent can build software, the **AI Foundation** must be defined.

This foundation defines:

- values
- behavior rules
- collaboration
- engineering standards
- learning loops

The foundation is implemented as **structured documents in the repository**.

Example directory:

```
/human-intent-os
/agent-foundation-os
/role-definition-os
/design-os
/build-os
/feedback-os
```

These layers correspond to the six development phases.

---

5. AI Foundation Development Phases

Phase 1 — Human Intent OS

Defines the **core philosophy of the AI organization**.

Includes:

- mission
- engineering principles
- design philosophy
- decision framework
- safety policies
- quality definitions

Example files:

```
human-intent-os/
 mission.md
 philosophy.md
 engineering-principles.md
 decision-framework.md
 quality-definition.md
 ethics-and-safety.md
 glossary.md
```

This layer acts as the **DNA of the AI system**.

Phase 2 — Agent Foundation OS

Defines **how agents behave during tasks**.

Includes:

- task lifecycle
- context ingestion
- memory rules
- communication protocol

- review process
- escalation policy

Example structure:

```
agent-foundation-os/
 runtime-model.md
 task-lifecycle.md
 communication-protocol.md
 reflection-loop.md
 escalation-policy.md
 orchestration-rules.md
```

This ensures agents operate consistently.

Phase 3 — Role Definition OS

Defines the **organizational roles for agents**.

Example roles:

- Product Strategist
- System Architect
- Software Engineer
- QA Reviewer
- DevOps Engineer

Example structure:

```
role-definition-os/

 product-strategist/
 system-architect/
 software-engineer/
```

```
 qa-reviewer/
 devops/
```

Each role includes:

```
role.md
responsibilities.md
boundaries.md
inputs.md
outputs.md
checklist.md
success-metrics.md
```

Phase 4 — Design OS

Transforms ideas into **system design artifacts**.

Design pipeline:

```
Idea
 ↓
Discovery
 ↓
Product Definition
 ↓
Domain Model
 ↓
User Flows
 ↓
System Context
 ↓
Architecture Blueprint
 ↓
API Contracts
```

```
 ↓
Implementation Readiness
```

Example directory:

```
design-os/
 discovery.md
 product-definition.md
 domain-model.md
 user-flows.md
 architecture-blueprint.md
 api-contracts.md
```

This prevents premature coding.

Phase 5 — Build Implementation OS

Defines **how software is generated and implemented**.

Includes:

- project generator
- repository templates
- coding standards
- testing rules
- CI/CD pipelines
- deployment rules

Example structure:

```
build-os/
 project-generator.md
 repository-template.md
 coding-standards.md
 testing-strategy.md
```

```
 ci-cd-pipeline.md
 deployment-policy.md
```

Templates for projects:

```
/templates
 nextjs-template
 backend-template
 fullstack-template
```

Phase 6 — Feedback & Learning OS

Allows the system to **learn from experience**.

Includes:

- task reflection
- failure analysis
- knowledge base
- improvement proposals
- agent performance metrics

Example structure:

```
feedback-os/
 task-reflection.md
 failure-analysis.md
 learning-database.md
 knowledge-library.md
 improvement-proposals.md
 governance-model.md
```

This enables continuous improvement.

---

6. AI Orchestration Layer

After the foundation is built, the next step is deploying the **AI Orchestrator**.

Example orchestrator:

```
OpenClow (AI Project Manager)
```

The orchestrator coordinates agents and workflows.

---

7. Dedicated AI Machine Setup

The orchestrator should run on a **dedicated machine**.

Recommended options:

- VPS
- dedicated server
- Mac Mini
- cloud VM

Example architecture:

```
Dedicated Machine (AI Control Server)
 ├ OpenClow Orchestrator
 ├ AI Agent Runtime
 ├ AI Models / APIs
 ├ Workflow Engine
 └ Repository Access
```

---

8. Secure Access Setup

The orchestrator must access **controlled resources only**.

Access setup:

Service	Purpose
GitHub	source code repository
Slack	communication
Storage	artifacts
CI/CD	build pipeline

Access should use:

```
service accounts
API tokens
restricted permissions
```

Avoid giving agents full system access.

---

9. Installing the AI Orchestrator

Example installation process:

Step 1 — Prepare Machine

Install:

```
docker
node
python
git
```

Step 2 — Install OpenClow

Example:

```
git clone orchestrator-repo
cd orchestrator
docker compose up
```

This launches the orchestrator service.

Step 3 — Connect AI Models

Connect LLM providers:

- OpenAI
- Claude
- local models

Add API keys.

Step 4 — Connect Repository

Configure access to the project repo containing the foundation.

Example:

```
git clone ai-software-factory
```

Agents will read these files as their **knowledge base**.

---

10. Installing AI Agents

Agents do not need separate machines.

They run inside the orchestrator.

Example agents:

```
product-agent
architect-agent
engineer-agent
qa-agent
devops-agent
```

Each agent uses:

```
foundation rules
role definition
design artifacts
project repository
```

---

11. Tooling for Agents

Agents may use development tools such as:

Tool	Purpose
Cursor	AI coding environment
Cloud Code	code generation
GitHub MCP	repository access
CI/CD tools	automated builds

These tools allow agents to **interact with the codebase**.

---

12. Project Workflow

Example development workflow:

```
Human requests feature
      ↓
Product Agent defines requirements
      ↓
Architect Agent designs system
      ↓
Engineer Agent generates code
      ↓
QA Agent reviews output
      ↓
DevOps Agent prepares deployment
```

This workflow repeats for every feature.

---

13. First Project Execution

Once the system is installed:

Step 1

Create project:

```
create-project
```

System asks:

```
Project name
Framework
Database
Auth provider
Testing framework
Deployment target
```

Step 2

Agents generate:

```
project repository
architecture
initial code
tests
deployment pipeline
```

Step 3

Develop iteratively.

Agents collaborate under orchestrator control.

---

14. Continuous Learning

After each project:

1. tasks are reviewed
2. failures are recorded
3. lessons added to knowledge base
4. foundation rules updated

This improves future projects.

---

15. Resulting System

At full maturity, the AI Software Factory can:

- create new software projects
- design architecture
- generate code
- detect defects
- review quality
- deploy applications
- accumulate engineering knowledge

The system becomes an **AI engineering organization**.

---

16. Final Architecture

```
Human
  ↓
AI Orchestrator
  ↓
AI Agent Organization
  ↓
```

```
Project Repository
  ↓
Software System
  ↓
Feedback & Learning
```

---

17. Key Principle

The power of this system is **not smarter AI models**.

The real advantage is:

organizational structure for AI engineering teams.

▼ Phase 1 - Human Intern

Great. We'll go **phase by phase** and start with **Phase 1: Human Intent OS**.

This phase is **the most important one**. If this layer is weak, all future agents will behave inconsistently. Think of this as **the DNA of your AI company**.

This phase should ideally produce **one core directory called** `human-intent-os/` that every future agent inherits.

Phase 1 — Human Intent OS

Goal:

Define the **core philosophy, values, and decision rules** that all AI agents must follow when building software.

This phase answers the question:

> *How should our AI think before it writes any code?*

You are not building agents yet.

You are defining **how agents must think**.

This is the **heart of your agentic system**.

Deliverables of Phase 1

By the end of Phase 1 you should have:

```
/human-intent-os
    mission.md
    philosophy.md
    engineering-principles.md
    design-principles.md
    decision-framework.md
    risk-policy.md
    collaboration-rules.md
    quality-definition.md
    ethics-and-safety.md
    glossary.md
```

These files define the **mental model** of your AI organization.

All future agents must inherit these.

Structure of Phase 1

Phase 1 contains **7 sub-steps**.

```
1 Define Mission
2 Define Philosophy
3 Define Engineering Principles
4 Define Design Principles
5 Define Decision Framework
6 Define Quality Definition
7 Define Collaboration Rules
```

Let's go through each.

Step 1 — Mission

Create:

```
human-intent-os/mission.md
```

Purpose: define **why the AI company exists**.

Example:

```
Mission

We build software systems that are:

- secure by default
- maintainable by design
- observable in production
- scalable by architecture
- understandable by humans
```

Agents must always align with this mission.

Example rule:

```
If a decision conflicts with the mission,
the mission overrides implementation convenience.
```

Step 2 — Philosophy

Create:

```
human-intent-os/philosophy.md
```

Purpose: define **how the AI thinks about software**.

Example philosophy sections:

Simplicity First

Prefer simple systems over complex systems.

Explicit Over Implicit

All assumptions must be documented.

Observability First

Every system must produce logs, metrics, and traces.

Security by Default

Security is not optional.

Spec Before Code

No implementation without a defined specification.

Step 3 — Engineering Principles

Create:

```
human-intent-os/engineering-principles.md
```

These define **how code should be written**.

Example principles:

Maintainability

Code must be understandable by another engineer within minutes.

Testability

Every feature must be testable.

Modularity

Systems should be composed of independent modules.

Fault Isolation

Failures must not cascade across systems.

Infrastructure Awareness

Software must consider deployment environment.

Example rule:

```
Agents must never introduce code
that cannot be tested or observed.
```

Step 4 — Design Principles

Create:

```
human-intent-os/design-principles.md
```

This governs **product and system design**.

Example:

Human-centered design

Interfaces should minimize cognitive load.

Consistency

Design patterns should be reused.

Progressive disclosure

Complexity should appear only when needed.

Performance awareness

Design must consider system performance early.

Step 5 — Decision Framework

Create:

```
human-intent-os/decision-framework.md
```

This teaches agents **how to choose between alternatives**.

Example structure:

```
Decision priority order:

1 Security
2 Correctness
3 Maintainability
4 Performance
5 Developer convenience
```

Example:

If performance conflicts with maintainability:

```
Prefer maintainability unless performance impact is critic
al.
```

Step 6 — Risk Policy

Create:

```
human-intent-os/risk-policy.md
```

This prevents agents from making unsafe choices.

Examples:

```
Never expose secrets in code
Never bypass authentication
```

```
Never remove audit logging
Never disable security controls
```

This file protects your system from dangerous agent behavior.

Step 7 — Collaboration Rules

Create:

```
human-intent-os/collaboration-rules.md
```

This defines how agents communicate.

Example protocol:

Every handoff must include:

```
Context
Goal
Constraints
Assumptions
Proposed solution
Open questions
Acceptance criteria
```

Agents cannot pass work without these.

Step 8 — Quality Definition

Create:

```
human-intent-os/quality-definition.md
```

Define what **"done" means**.

Example:

A feature is complete only when:

```
specification exists
code implemented
tests written
documentation updated
observability added
security reviewed
```

Step 9 — Ethics & Safety

Create:

```
human-intent-os/ethics-and-safety.md
```

This ensures agents behave responsibly.

Example:

```
Do not generate harmful software
Do not produce malicious code
Do not assist in unethical system design
```

Step 10 — Glossary

Create:

```
human-intent-os/glossary.md
```

Define shared vocabulary.

Example:

```
Agent: an AI role performing a specialized task
Specification: structured definition of system behavior
```

```
Architecture: high-level system design
Implementation: code that realizes a specification
```

This avoids misunderstandings between agents.

Example Directory

Final Phase 1 structure:

```
human-intent-os/
│
├ mission.md
├ philosophy.md
├ engineering-principles.md
├ design-principles.md
├ decision-framework.md
├ risk-policy.md
├ collaboration-rules.md
├ quality-definition.md
├ ethics-and-safety.md
└ glossary.md
```

Completion Criteria for Phase 1

Phase 1 is finished when:

✔️ Every document exists

✔️ Each document has clear rules

✔️ No contradictions between principles

✔️ All agents must reference this layer

What happens after Phase 1?

Phase 2 will build:

Agent Foundation OS

This layer will define:

- task lifecycle
- agent memory
- reflection loops
- agent communication
- system prompts
- task orchestration
- agent runtime behavior

Phase 1 defines **how agents think**.

Phase 2 defines **how agents operate**.

One Important Suggestion

Do **not overcomplicate Phase 1**.

These documents should be:

- clear
- short
- strict
- actionable

Agents cannot reason with vague philosophy.

Question for You (important)

Before we move to **Phase 2**, I want to understand something critical.

In your system architecture:

Will your agents run primarily inside:

1️⃣ **LLM prompt frameworks (like Cursor / Claude / GPT)**

2️⃣ **Code agents running inside your repo**

3️⃣ **Cloud orchestrated agents (OpenAI / LangGraph / CrewAI / etc)**

4️⃣ **Custom internal orchestration**

Because **Phase 2 architecture changes a lot depending on this choice.**

Once you answer this, I will build **Phase 2 — Agent Foundation OS** in detail.

▼ Phase 2 - Agent Foundation

Perfect. We continue with **Phase 2: Agent Foundation OS**.

Phase 1 defined **how your AI company thinks**.

Phase 2 defines **how every agent behaves at runtime**.

This is the layer that turns philosophy into an actual operating system for agents.

It also lines up well with the repo structure you shared: Russell's AI Software Factory already separates a `foundation/` layer from downstream operating layers, and is meant to be used as a template plus command-driven workflow for real projects. (GitHub)

Phase 2 — Agent Foundation OS

Main goal:

Create the shared runtime contract that all agents must follow before you make any specialist role.

This phase answers:

> "Now that we know what the AI should believe, how should it actually work?"

By the end of this phase, your agents should all behave consistently in:

- task intake
- context reading

- planning
- execution
- handoff
- self-checking
- reflection
- escalation

This is the phase where your “heart” from Human Intent becomes a **nervous system**.

What Phase 2 should produce

You should end Phase 2 with one directory like this:

```
/agent-foundation-os
  runtime-model.md
  task-lifecycle.md
  context-ingestion.md
  memory-policy.md
  communication-protocol.md
  handoff-contract.md
  reflection-loop.md
  escalation-policy.md
  output-contracts.md
  review-checklist.md
  orchestration-rules.md
  tool-usage-policy.md
```

If Phase 1 is the constitution, Phase 2 is the daily operating manual.

The purpose of Phase 2

Without this layer, each agent will:

- interpret tasks differently
- communicate differently
- forget important context
- produce uneven output quality
- drift in behavior over time

With this layer, every specialist agent inherits the same base behavior.

That means your future:

- architect agent
- backend agent
- frontend agent
- QA agent
- DevOps agent

...will all act like members of the same company, not random freelancers.

Phase 2 sub-phases

I would split Phase 2 into 8 parts:

1. Runtime Identity
2. Task Lifecycle
3. Context Ingestion
4. Memory and Learning
5. Communication and Handoff
6. Reflection and Review
7. Escalation and Safety Boundaries
8. Orchestration and Tooling Rules

---

1. Runtime Identity

Create:

```
agent-foundation-os/runtime-model.md
```

This file defines what an agent fundamentally is in your system.

It should answer:

- What is an agent?
- What does an agent own?
- What can an agent decide?
- What can an agent not decide?
- What is its relationship to human users?
- What is its relationship to other agents?

What to define

Agent identity

Each agent is:

- goal-driven
- context-dependent
- role-constrained
- reviewable
- non-sovereign

That last one matters:

the agent is **never the final authority**.

Core rule

Agents may propose, analyze, generate, and review.

Humans approve strategic direction, policy change, and high-risk decisions.

Agent categories

At foundation level, define 3 universal categories:

- **Thinkers**: analyze, plan, reason
- **Builders**: generate implementation artifacts
- **Reviewers**: evaluate outputs against standards

Later, one real agent can combine multiple categories, but the foundation should separate them conceptually.

Why this matters

This stops role confusion later.

For example:

- architect proposes structure
- builder implements
- reviewer checks compliance

Even if one LLM does all three in sequence, the **mode switching** becomes explicit.

---

2. Task Lifecycle

Create:

```
agent-foundation-os/task-lifecycle.md
```

This is one of the most important files in the whole system.

Every agent task should follow the same lifecycle:

```
1 Receive task
2 Parse goal
3 Read required context
```

```
4 Identify constraints
5 Draft plan
6 Execute
7 Self-review
8 Handoff or escalate
9 Reflect and log lessons
```

Required lifecycle stages

Stage A — Intake

The agent receives a task.

It must identify:

- requested outcome
- expected artifact
- success criteria
- urgency
- risk level

Stage B — Understanding

Before doing anything, the agent must classify:

- Is this a discovery task?
- Is this a design task?
- Is this an implementation task?
- Is this a review task?
- Is this a debugging task?

This matters because different task types need different behavior.

Stage C — Context loading

The agent must read only the minimum required context, but enough to avoid hallucination.

Required context layers:

1. Human Intent OS
2. Agent Foundation OS
3. Role OS
4. Project-specific docs
5. Codebase context
6. Current task context

Stage D — Plan first

Before producing a large output, the agent should generate an internal task plan.

For example:

- objective
- assumptions
- dependencies
- output structure
- risk checks

Stage E — Execution

The agent performs the task using the approved scope.

Stage F — Self-review

Every output must pass a pre-handoff checklist.

Stage G — Handoff / escalation

If complete, hand off.

If blocked, escalate with structured explanation.

Stage H — Reflection

The system logs:

- what happened
- what was missing
- what should improve next time

---

3. Context Ingestion

Create:

```
agent-foundation-os/context-ingestion.md
```

This file defines how agents consume information.

A weak agent loads context randomly.

A mature agent reads context in layers.

Recommended context order

Layer 1 — Global laws

Human Intent OS

Layer 2 — Shared operating behavior

Agent Foundation OS

Layer 3 — Role rules

Specialist role OS

Layer 4 — Project truth

Specs, ADRs, architecture docs, PRDs, design docs

Layer 5 — Runtime truth

Current code, tests, configs, logs, issues, tickets

Layer 6 — Current request

The immediate task from user or another agent

Context ingestion rules

Rule 1

Do not start implementation if the relevant project truth is missing.

Rule 2

Do not rely only on the latest prompt if the codebase or spec may contradict it.

Rule 3

When context conflicts, apply this priority:

1. human-approved project truth
2. human intent rules
3. role rules
4. latest request
5. agent assumptions

Rule 4

Agents must explicitly list assumptions when context is incomplete.

This one is critical.

You want the agent to say:

- “I found no migration policy doc, so I am assuming backward compatibility is required.”

Not silently guess.

---

4. Memory and Learning

Create:

```
agent-foundation-os/memory-policy.md
agent-foundation-os/reflection-loop.md
```

This is where your “nurturing” idea becomes real.

An agent becomes better not because it has a poetic identity, but because it has a disciplined memory and reflection loop.

Memory types to define

Working memory

Only for the current task.

Contains:

- current objective
- current constraints
- temporary reasoning state
- intermediate outputs

Project memory

Persistent within one project.

Contains:

- architecture decisions
- conventions
- common pitfalls
- domain language
- accepted patterns

Organizational memory

Persistent across projects.

Contains:

- reusable standards
- anti-patterns
- common review failures
- baseline templates
- security lessons

Reflection loop design

After each substantial task, log:

- task type
- output produced
- success/failure
- missing context
- recurring issues
- rule updates suggested

Suggested reflection template

```
Task:
Expected outcome:
Actual outcome:
What went well:
What failed:
Why it failed:
Which rule was missing or weak:
What should be added to foundation or role OS:
```

Important boundary

Do not let agents rewrite core rules automatically.

Use:

- proposed rule changes
- human review
- approved updates

That prevents uncontrolled drift.

---

5. Communication and Handoff

Create:

```
agent-foundation-os/communication-protocol.md
agent-foundation-os/handoff-contract.md
```

This is where multi-agent teamwork becomes reliable.

Without this, agents throw vague outputs at one another.

Every handoff should include

```
Task summary
Current context
What was completed
What remains open
Assumptions made
Known risks
Required next action
Acceptance criteria
```

Handoff example

A product agent handing to an architect should say:

```
Goal:
Create a multi-tenant visitor management system.
```

```
Completed:
PRD draft, user roles, core workflows.

Constraints:
Supabase backend, audit logging mandatory, hospital multi-
tenant isolation required.

Open issues:
Unsure whether approval flow belongs in app logic or workf
low engine.

Next expected output:
Architecture proposal with trade-offs.

Acceptance criteria:
Must cover auth, tenancy boundaries, audit logging, deploy
ment assumptions.
```

That format is simple, but extremely powerful.

Communication rules

Rule 1

Do not hand off unresolved ambiguity without explicitly labeling it.

Rule 2

Do not hand off conclusions without supporting context.

Rule 3

Do not ask another agent to infer hidden assumptions that you already know.

Rule 4

Every handoff must be testable by the receiving agent.

6. Reflection and Review

Create:

```
agent-foundation-os/review-checklist.md
```

This file defines how agents review themselves and each other.

Universal review dimensions

Every output should be checked for:

- alignment with goal
- compliance with Human Intent OS
- completeness
- correctness
- clarity
- maintainability
- risk exposure
- next-step usability

Minimum self-review checklist

Before finalizing, an agent must ask:

1. Did I solve the requested problem?
2. Did I use the right context?
3. Did I invent anything without labeling it as assumption?
4. Did I violate any core principle?
5. Is my output usable by the next actor?
6. Did I leave hidden risk unmentioned?
7. Would another agent or human understand my reasoning and output?

This is where your agents start feeling disciplined instead of flashy.

---

7. Escalation and Safety Boundaries

Create:

```
agent-foundation-os/escalation-policy.md
agent-foundation-os/tool-usage-policy.md
```

This defines when an agent must stop, escalate, or narrow scope.

Escalation triggers

The agent must escalate when:

- the task conflicts with core policy
- required context is missing for a high-risk decision
- multiple reasonable paths exist with business impact
- security/privacy/compliance risk is present
- tool output conflicts with system truth
- the agent is being asked to override governance rules

Escalation output format

```
Issue:
Why blocked:
Risk if continued:
What information is needed:
Recommended options:
```

Tool usage policy

You mentioned cloud-based code tooling and eventually plugging this into a real implementation environment.

So define these tool classes now:

Read tools

Repo browsing, file reading, docs reading, logs, ticket systems

Write tools

Create file, edit file, update docs, scaffold project, modify configs

Execute tools

Run tests, run lint, build app, run migration, deploy preview

Dangerous tools

Delete data, modify production config, apply DB migration, rotate secrets, disable controls

Core policy

Agents may:

- freely use read tools
- cautiously use write tools
- use execute tools only within defined environments
- never use dangerous tools without explicit human-approved policy

This matters a lot once your foundation leaves markdown and touches real code.

---

8. Orchestration and Tooling Rules

Create:

```
agent-foundation-os/orchestration-rules.md
```

This file explains how agents are composed into workflows.

Given the AI Software Factory template uses a phase-driven sequence and command-based operation around shared documents, your own system should also treat agents as participants in a controlled workflow rather than as free-roaming autonomous workers. (GitHub)

Recommended orchestration pattern for your stage

Do **not** start with a swarm.

Start with a **linear orchestrated pipeline**:

```
Human → Product/Intent Agent → Architect Agent → Builder A
gent → Reviewer Agent → Human
```

Later you can support loops:

- reviewer back to builder
- architect back to product
- QA back to builder

But phase 2 should define the simple default pipeline first.

Why this is best now

Because your current challenge is foundation maturity, not autonomy complexity.

A swarm sounds powerful, but it introduces:

- duplicated reasoning
- context fragmentation
- conflicting outputs
- coordination overhead

You want discipline first.

Suggested orchestration modes

Mode 1 — Guided mode

Human supervises every major step.

Best for early adoption.

Mode 2 — Delegated mode

Agent chain works through a bounded workflow, then reports back.

Best for mature internal projects.

Mode 3 — Review-first mode

Builder proposes, reviewer checks before human sees it.

Best for code-intensive tasks.

Start with Guided mode.

What “done” looks like for Phase 2

Phase 2 is complete when:

- every agent has one shared runtime contract
- every task follows one lifecycle
- every handoff uses one standard structure
- every agent knows when to escalate
- every output goes through self-review
- memory and reflection are defined
- orchestration is simple and controlled

At that point, you do **not** yet have specialist intelligence, but you do have **behavioral consistency**.

That is exactly what you need before creating specialist agents.

Relationship between Phase 1 and Phase 2

A simple way to remember it:

- **Phase 1 = Why we think this way**
- **Phase 2 = How we behave while working**

Phase 1 without Phase 2 becomes philosophy theater.

Phase 2 without Phase 1 becomes efficient chaos.

You need both.

What comes next after Phase 2

Then we move into **Phase 3: Role Definition OS**.

That phase will define:

- which specialist agents exist
- what each role owns
- what each role must never own
- role-specific inputs/outputs
- role-specific checklists
- role-specific success criteria
- role-to-role collaboration map

That is the phase where your first real “children” get their professions.

If you want, I’ll continue directly with **Phase 3 — Role Definition OS**.

▼ Phase 3 - Specialised Agentic OS

▼ Phase 4 - System Design OS

Excellent — now we enter **Phase 4: Design OS**.

This is one of the most important phases because it **bridges thinking → building**.

Up to now:

- **Phase 1 — Human Intent OS** → defines values and philosophy
- **Phase 2 — Agent Foundation OS** → defines agent behavior
- **Phase 3 — Role Definition OS** → defines who does what

Now we introduce the **Design Layer**.

This phase answers the critical question:

> *How does an idea become a buildable system before code exists?*

This is exactly the purpose of **Design OS** in Brian Casel's framework — a structured layer between product thinking and engineering implementation.

Without this layer, teams jump straight into coding and create **architecture chaos**.

Phase 4 — Design OS

Goal

Transform product intent into **clear system design artifacts** that engineers can implement safely.

This phase ensures:

- architecture clarity
- system boundaries
- user flow understanding
- data structure definition
- integration mapping
- operational assumptions

Before any coding begins.

What Phase 4 should produce

Directory:

```
/design-os
    discovery.md
    product-definition.md
    domain-model.md
    user-flows.md
    system-context.md
    architecture-blueprint.md
    integration-map.md
    data-model.md
    api-contracts.md
    technical-risk-assessment.md
    implementation-readiness.md
```

These artifacts become the **design contract**.

Your engineering agents cannot start building until these documents exist.

The Design Pipeline

The design phase itself has a **workflow pipeline**.

```
Idea
 ↓
Discovery
 ↓
Product Definition
 ↓
Domain Model
 ↓
User Flows
 ↓
```

```
System Context
 ↓
Architecture Blueprint
 ↓
Integration Map
 ↓
Data Model
 ↓
API Contracts
 ↓
Implementation Readiness
```

Each stage is owned by specific agents.

Step 1 — Discovery

Create:

```
design-os/discovery.md
```

This document captures the **raw idea and its context**.

Purpose:

Understand the problem before defining a solution.

Contents:

- problem statement
- user types
- current pain points
- environment constraints
- stakeholders
- expected outcomes

Example:

```
Problem:
Hospitals require a visitor management system with strict
audit logging and approval flows.

Users:
- visitor
- security officer
- admin

Constraints:
- must support multi-tenant hospitals
- must maintain audit trails
- must integrate with badge printing

Outcome:
Secure and traceable visitor management workflow.
```

Owner: **Product Strategist**

Step 2 — Product Definition

Create:

```
design-os/product-definition.md
```

This is the **PRD layer**.

Define:

- product scope
- core features
- non-goals
- success criteria
- user stories

- acceptance criteria

Example user story:

```
As a hospital security officer,
I want to approve visitor requests,
so that only authorized visitors can enter.
```

Owner: **Product Strategist**

Step 3 — Domain Model

Create:

```
design-os/domain-model.md
```

This is extremely important.

It defines the **language of the system**.

Example entities:

```
Visitor
VisitRequest
Approval
Badge
SecurityOfficer
AuditLog
```

Define relationships:

```
Visitor → VisitRequest
VisitRequest → Approval
VisitRequest → Badge
Approval → SecurityOfficer
```

This step prevents inconsistent naming across the system.

Owner: **System Architect**

Step 4 — User Flows

Create:

```
design-os/user-flows.md
```

Describe **how users interact with the system**.

Example flow:

```
Visitor registers
 ↓
Request created
 ↓
Security officer review
 ↓
Approval granted
 ↓
Badge printed
 ↓
Visitor enters facility
```

This clarifies behavior before architecture.

Owner: **Product Strategist + Architect**

Step 5 — System Context

Create:

```
design-os/system-context.md
```

Define how the system fits into the **external ecosystem**.

Example:

```
External systems:
- identity provider
- badge printer
- hospital directory
- audit logging system
```

Draw boundaries:

```
[Visitor App]
      ↓
[Visitor Management System]
      ↓
[Badge Printing Service]
```

Owner: **System Architect**

Step 6 — Architecture Blueprint

Create:

```
design-os/architecture-blueprint.md
```

This defines the **high-level system structure**.

Example:

```
Frontend
  - Next.js

Backend
  - Supabase

Services
  - approval workflow
  - audit service
```

```
  - notification service

Storage
  - PostgreSQL
```

Also define:

- service boundaries
- module responsibilities
- scalability assumptions

Owner: **System Architect**

Step 7 — Integration Map

Create:

```
design-os/integration-map.md
```

Define how systems talk to each other.

Example:

```
Visitor App → API Gateway
API Gateway → Auth Service
Auth Service → Supabase
Approval Service → Notification Service
```

Owner: **System Architect**

Step 8 — Data Model

Create:

```
design-os/data-model.md
```

Define database structure.

Example:

```
visitors
visit_requests
approvals
badges
audit_logs
```

Define relationships:

```
visit_requests.visitor_id → visitors.id
approvals.request_id → visit_requests.id
```

Owner: **Architect + Database Engineer**

Step 9 — API Contracts

Create:

```
design-os/api-contracts.md
```

Define APIs before code.

Example:

```
POST /visit-request
GET /visit-request/{id}
POST /approve-request
GET /visitor-history
```

Define request/response schemas.

This prevents frontend/backend misalignment.

Owner: **System Architect**

Step 10 — Technical Risk Assessment

Create:

```
design-os/technical-risk-assessment.md
```

Identify risks early.

Example:

```
Risk: multi-tenant data leakage
Mitigation: strict tenant isolation

Risk: printer integration failure
Mitigation: retry queue
```

Owner: **Architect + QA**

Step 11 — Implementation Readiness

Create:

```
design-os/implementation-readiness.md
```

Final checkpoint.

Verify:

- architecture approved
- API defined
- data model defined
- flows validated
- risks identified

Only after this document exists can the **engineering phase start**.

Owner: **Architect + QA**

What Phase 4 Achieves

This phase ensures:

✔️ ideas become structured designs

✔️ architecture is clear

✔️ APIs are defined

✔️ data structures exist

✔️ risks are identified

Before code exists.

This dramatically reduces engineering chaos.

When Phase 4 is finished

You now have a **design-ready system**.

Only then can Phase 5 begin.

Phase 5 Preview — Build Implementation OS

This next phase will define:

- code scaffolding
- project generators
- repo templates
- coding standards
- test pipelines
- CI/CD pipelines
- agent-assisted coding workflows

This is where your **AI foundation starts generating real software**.

This phase will connect directly with what you mentioned earlier:

> when someone pulls our boilerplate repository and starts a project

Phase 5 will define exactly how that works.

If you want, next I will show you **Phase 5 — Build Implementation OS**, where your AI agents begin **generating real projects automatically**.

▼ Phase 5 - Implementation Source of Truth OS

Excellent — now we reach **Phase 5: Build Implementation OS**.

This is where your entire foundation finally **touches real code**.

Until now you designed:

- **Phase 1:** Human Intent (beliefs and philosophy)
- **Phase 2:** Agent Foundation (behavior and runtime rules)
- **Phase 3:** Role Definition (who does what)
- **Phase 4:** Design OS (how ideas become system design)

Now Phase 5 answers:

> *How do agents actually generate and build real software?*

This is exactly the step you mentioned earlier about:

- pulling a **boilerplate repo**
- asking configuration questions
- generating a **project scaffold**
- letting agents continue development

This phase turns your AI system into a **software factory**.

Phase 5 — Build Implementation OS

Goal

Create a **standardized software construction pipeline** that allows AI agents to:

1. initialize a new project
2. scaffold architecture
3. implement features
4. enforce standards
5. run tests
6. prepare deployment

All automatically guided by your foundation.

What Phase 5 should produce

Directory structure:

```
/build-os
    project-generator.md
    project-config-schema.md
    repository-template.md
    coding-standards.md
    code-review-rules.md
    testing-strategy.md
    migration-policy.md
    dependency-policy.md
    ci-cd-pipeline.md
    deployment-policy.md
```

And a real **code layer**:

```
/templates
    nextjs-template
    backend-template
    fullstack-template
    microservice-template
```

These templates become the **starting point for every project**.

Build Pipeline Overview

The build process follows this sequence:

```
Project Initialization
      ↓
Stack Configuration
      ↓
Template Generation
      ↓
Architecture Scaffold
      ↓
Feature Implementation
      ↓
Automated Testing
      ↓
Quality Review
      ↓
Deployment Preparation
```

Each step activates different agents.

Step 1 — Project Generator

Create:

```
build-os/project-generator.md
```

This defines how a **new project begins**.

This is the CLI / automated workflow you mentioned.

Example flow when someone creates a project:

```
create-project
```

The system asks:

```
Project name?
Application type?
Frontend framework?
Backend system?
Database type?
Authentication?
Testing framework?
Deployment target?
```

Example:

```
Project name: visitor-management
Frontend: Next.js
Backend: Supabase
Auth: Supabase Auth
Testing: Jest
Deployment: Vercel
```

Your AI agents will use these answers to configure the project.

Step 2 — Project Configuration Schema

Create:

```
build-os/project-config-schema.md
```

This defines the configuration structure.

Example:

```
{
  "project_name": "",
  "app_type": "",
  "frontend": "",
  "backend": "",
  "database": "",
  "auth_provider": "",
  "testing_framework": "",
  "deployment_target": ""
}
```

This schema drives the template generator.

Step 3 — Repository Template

Create:

```
build-os/repository-template.md
```

Define the default repo structure.

Example:

```
/app
/components
/services
/hooks
/lib
/types
/tests
```

```
/docs
/config
/scripts
```

Include:

- lint configuration
- formatting rules
- base environment setup
- logging
- monitoring hooks

This ensures **every project begins consistently**.

Step 4 — Architecture Scaffold

After generating the repo, the system architect agent creates the architecture layout.

Example:

```
src/
  modules/
      visitor
      approval
      badge
  services/
      auth
      notifications
      audit
```

The scaffold follows the **design artifacts from Phase 4**.

This prevents architecture drift.

Step 5 — Coding Standards

Create:

```
build-os/coding-standards.md
```

Define:

- naming conventions
- folder structures
- function design
- error handling
- logging rules
- API patterns

Example rules:

```
Functions must be pure where possible
Services must not depend on UI modules
Database queries must be isolated
Errors must produce structured logs
```

Agents must follow these rules when generating code.

Step 6 — Code Review Rules

Create:

```
build-os/code-review-rules.md
```

Your QA agents use this.

Review dimensions:

```
Correctness
Architecture compliance
Security
Maintainability
Test coverage
Performance risk
```

Review output example:

```
PASS / FAIL
Critical Issues
Major Issues
Minor Improvements
Suggested Refactors
```

Step 7 — Testing Strategy

Create:

```
build-os/testing-strategy.md
```

Define your testing layers.

Example:

```
Unit Tests
Integration Tests
API Tests
UI Tests
```

Coverage rule example:

```
Critical business logic must have unit tests
API endpoints must have integration tests
```

```
Authentication flows must have security tests
```

Agents must generate tests alongside code.

Step 8 — Migration Policy

Create:

```
build-os/migration-policy.md
```

This protects database evolution.

Rules:

```
Never modify production schema without migration
All migrations must be reversible
All migrations must include rollback instructions
```

This is very important if your agents generate DB schema changes.

Step 9 — Dependency Policy

Create:

```
build-os/dependency-policy.md
```

Agents must not randomly add packages.

Rules:

```
Prefer native libraries first
Avoid heavy dependencies
Dependencies must have active maintenance
Security vulnerabilities must be checked
```

This prevents dependency bloat.

Step 10 — CI/CD Pipeline

Create:

```
build-os/ci-cd-pipeline.md
```

Example pipeline:

```
Install dependencies
Run lint
Run tests
Build application
Run security checks
Deploy preview
```

Agents must ensure all projects support this.

Step 11 — Deployment Policy

Create:

```
build-os/deployment-policy.md
```

Define environments:

```
development
staging
production
```

Rules:

```
All deployments must pass CI
Database migrations must run safely
Rollback must be possible
```

Example of the Full Build Flow

Imagine a developer starts a new project.

Step 1

```
npx create-ai-project
```

Step 2

System asks configuration.

Step 3

Templates generated.

Step 4

Design OS artifacts loaded.

Step 5

Architect agent scaffolds system.

Step 6

Engineer agent implements features.

Step 7

QA agent reviews code.

Step 8

CI/CD runs automatically.

Step 9

Deployment prepared.

What Phase 5 Achieves

After this phase:

Your AI system can:

- start new projects

- generate architectures
- scaffold repos
- enforce standards
- implement features
- test code
- prepare deployment

This is the first moment your **AI software factory becomes real**.

When Phase 5 is complete

You now have:

```
Idea → Design → Code → Test → Deploy
```

Fully structured.

Phase 6 Preview — Feedback & Scaling OS

The final phase introduces:

- learning loops
- agent performance metrics
- system improvement
- knowledge accumulation
- scaling your AI workforce

This is where your AI agents become **better over time**.

If you'd like, the next step I can show you is **Phase 6 — Feedback, Learning, and Scaling OS**, which is the phase that truly makes your **AI agents evolve and mature like the "children" idea you described.**

▼ Phase 6 - Scaling and Governance

Excellent. We now reach the **final phase: Phase 6 — Feedback, Learning, and Scaling OS**.

This is the phase that transforms your system from **a static AI factory** into **a self-improving AI organization**.

Until now, you built:

1. **Human Intent OS** → belief system
2. **Agent Foundation OS** → behavior rules
3. **Role Definition OS** → organizational structure
4. **Design OS** → thinking before building
5. **Build Implementation OS** → generating software

Now Phase 6 answers:

> How does the system **improve itself over time**?

This is where your idea of **raising AI like children** actually becomes real.

Because learning is not just knowledge — it is **experience accumulation**.

Phase 6 — Feedback, Learning, and Scaling OS

Goal

Create systems that allow your AI agents to:

- learn from past work
- improve decision quality
- accumulate engineering knowledge
- refine their behavior
- scale across many projects

- become progressively more capable

This phase transforms your system from:

```
AI Tools
```

into

```
AI Organization
```

What Phase 6 should produce

Directory:

```
/feedback-os
    task-reflection.md
    learning-database.md
    agent-performance.md
    improvement-proposals.md
    failure-analysis.md
    knowledge-library.md
    scaling-policy.md
    governance-model.md
```

This layer becomes the **memory and evolution engine** of your AI system.

The Feedback Loop

The core loop looks like this:

```
Task Execution
      ↓
Output Produced
      ↓
Review & QA
```

```
      ↓
Reflection
      ↓
Learning Update
      ↓
Foundation Improvement
```

This loop repeats across **every project and every task**.

Step 1 — Task Reflection

Create:

```
feedback-os/task-reflection.md
```

Every significant task should produce a **reflection log**.

Reflection template:

```
Task Description
Expected Outcome
Actual Outcome
Success / Failure
Issues Encountered
Missing Context
Unexpected Challenges
Improvement Suggestions
```

Example:

```
Task:
Implement visitor approval workflow

Expected:
Workflow with audit logging
```

```
Actual:
Workflow built but audit logs missing

Failure Reason:
Audit logging requirement not detected in architecture lay
er

Suggested Improvement:
Add mandatory audit logging rule in architecture checklist
```

This is extremely powerful.

Because now your system **captures experience**.

Step 2 — Failure Analysis

Create:

```
feedback-os/failure-analysis.md
```

This document categorizes failures.

Example categories:

```
Requirement misunderstanding
Architecture flaw
Implementation error
Missing tests
Security oversight
Performance regression
Dependency misconfiguration
```

Each failure must be classified.

Example:

```
Failure:
Tenant isolation bug

Category:
Architecture flaw

Root Cause:
Shared query logic without tenant filter

Fix:
Introduce tenant middleware layer
```

Over time, this builds a **failure knowledge base**.

Step 3 — Learning Database

Create:

```
feedback-os/learning-database.md
```

This becomes your **AI organization's institutional knowledge**.

Store lessons such as:

```
Common bugs
Security vulnerabilities
Architecture pitfalls
Integration failures
Deployment mistakes
Performance bottlenecks
```

Example entry:

```
Lesson:
Avoid direct DB queries in API routes.
```

```
Reason:
Breaks architecture layering and causes maintainability is
sues.

Rule Update:
All DB access must go through service layer.
```

This allows future agents to **avoid repeating mistakes**.

Step 4 — Knowledge Library

Create:

```
feedback-os/knowledge-library.md
```

This stores reusable best practices.

Examples:

```
Authentication patterns
Multi-tenant isolation strategies
API versioning strategies
Caching strategies
Observability setups
Database indexing strategies
```

Example:

```
Topic:
Multi-tenant architecture

Recommended Pattern:
Tenant ID column + row-level security

Risks:
```

```
Cross-tenant data leaks

Mitigation:
Mandatory tenant filter middleware
```

Over time, this becomes your **engineering handbook**.

Step 5 — Agent Performance Metrics

Create:

```
feedback-os/agent-performance.md
```

Define metrics to evaluate agents.

Example metrics:

```
Task completion accuracy
Architecture correctness
Defect detection rate
Test coverage quality
Implementation rework rate
Review effectiveness
```

Example measurement:

```
Software Engineer Agent

Tasks completed: 42
Defects found in QA: 5
Rework rate: 12%
Test coverage average: 85%
```

This helps identify:

- weak agents

- improvement opportunities
- areas needing new rules

Step 6 — Improvement Proposals

Create:

```
feedback-os/improvement-proposals.md
```

Agents can propose improvements to:

- Human Intent OS
- Agent Foundation OS
- Role Definition OS
- Design OS
- Build OS

Example:

```
Proposal:
Add database migration validation step.

Reason:
Several recent tasks introduced schema conflicts.

Suggested Change:
Add migration validation to CI pipeline.
```

Humans review these proposals.

Approved proposals update the system.

Step 7 — Knowledge Accumulation

This is where your system becomes **powerful over time**.

Your AI organization should accumulate:

```
Architectural patterns
Common system designs
Deployment templates
Testing patterns
Security patterns
```

These become **accelerators for future projects**.

Example:

```
Reusable blueprint:
Visitor Management System

Components:
Auth
Visitor Registry
Approval Workflow
Badge Printing
Audit Logging
```

Next time a similar project appears, your AI system can start **80% complete**.

Step 8 — Scaling Policy

Create:

```
feedback-os/scaling-policy.md
```

Define how your AI organization expands.

Scaling methods:

```
More agents
More roles
More specialized knowledge
More reusable templates
More automation
```

Example progression:

```
Stage 1
4 agents

Stage 2
8 agents

Stage 3
12 agents

Stage 4
Full AI software organization
```

Scaling should be gradual.

Step 9 — Governance Model

Create:

```
feedback-os/governance-model.md
```

Define **who controls the system**.

Important rule:

```
AI can propose improvements
Humans approve system changes
```

Governance layers:

```
Human Leadership
↓
AI System Rules
↓
Agent Behavior
↓
Project Execution
```

This prevents uncontrolled AI drift.

Final Architecture of Your System

At the end of all 6 phases, your system structure becomes:

```
/human-intent-os
/agent-foundation-os
/role-definition-os
/design-os
/build-os
/feedback-os
```

Together they form your **AI Software Organization**.

What Your System Will Be Able to Do

Once fully implemented, your AI organization can:

- start new software projects
- design architecture
- generate code
- review implementation
- detect bugs

- learn from mistakes
- accumulate knowledge
- improve future projects

This is essentially a **self-improving AI software factory**.

The Big Picture

You started with this idea:

> Raise AI agents like children that become skilled engineers.

What we built here is the **education system** for those children.

```
Human Intent → values
Foundation OS → behavior
Role OS → profession
Design OS → thinking
Build OS → creating
Feedback OS → learning
```

That is exactly how real organizations mature.

One Final Insight

Your real advantage will not be:

better prompts

It will be:

better organizational design for AI systems.

Most teams try to make a **smart AI**.

You are building something much stronger:

```
A structured AI engineering organization
```

If you want, the next thing I can show you is something extremely useful:

How to implement this entire architecture in a real repository structure that works with tools like Cursor, Claude, or other AI coding systems.

That would turn this whole framework into a **practical AI Software Factory you can run immediately.**

▼ Implementation Guideline

1. Recommended GitHub Repository Structure

I suggest you keep this as **one main repository first**, not many repos yet.

Example repo name:

```
ai-software-factory
```

Recommended structure:

```
ai-software-factory/
│
├─ README.md
├─ ROADMAP.md
├─ GOVERNANCE.md
├─ SECURITY.md
├─ CONTRIBUTING.md
├─ .env.example
├─ .gitignore
│
├─ docs/
│  ├─ overview/
│  │  ├─ vision.md
│  │  ├─ business-flow.md
│  │  └─ terminology.md
```

```
│   │
│   ├─ architecture/
│   │   ├─ system-architecture.md
│   │   ├─ deployment-architecture.md
│   │   ├─ orchestration-flow.md
│   │   └─ security-boundaries.md
│   │
│   ├─ operations/
│   │   ├─ setup-vps.md
│   │   ├─ setup-orchestrator.md
│   │   ├─ setup-model-providers.md
│   │   ├─ setup-github-access.md
│   │   ├─ setup-mcp.md
│   │   └─ incident-runbook.md
│   │
│   └─ examples/
│       ├─ example-project-init.md
│       ├─ example-agent-handoff.md
│       └─ example-review-loop.md
│
├─ foundation/
│   ├─ human-intent-os/
│   │   ├─ mission.md
│   │   ├─ philosophy.md
│   │   ├─ engineering-principles.md
│   │   ├─ design-principles.md
│   │   ├─ decision-framework.md
│   │   ├─ risk-policy.md
│   │   ├─ collaboration-rules.md
│   │   ├─ quality-definition.md
│   │   ├─ ethics-and-safety.md
│   │   └─ glossary.md
│   │
│   ├─ agent-foundation-os/
│   │   ├─ runtime-model.md
│   │   ├─ task-lifecycle.md
```

```
│   │   ├─ context-ingestion.md
│   │   ├─ memory-policy.md
│   │   ├─ communication-protocol.md
│   │   ├─ handoff-contract.md
│   │   ├─ reflection-loop.md
│   │   ├─ escalation-policy.md
│   │   ├─ output-contracts.md
│   │   ├─ review-checklist.md
│   │   ├─ orchestration-rules.md
│   │   └─ tool-usage-policy.md
│   │
│   ├─ role-definition-os/
│   │   ├─ role-map.md
│   │   ├─ collaboration-map.md
│   │   ├─ role-selection-policy.md
│   │   │
│   │   ├─ product-strategist/
│   │   │   ├─ role.md
│   │   │   ├─ responsibilities.md
│   │   │   ├─ boundaries.md
│   │   │   ├─ inputs.md
│   │   │   ├─ outputs.md
│   │   │   ├─ checklist.md
│   │   │   └─ success-metrics.md
│   │   │
│   │   ├─ system-architect/
│   │   ├─ software-engineer/
│   │   ├─ qa-reviewer/
│   │   └─ devops-platform/
│   │
│   ├─ design-os/
│   │   ├─ discovery.md
│   │   ├─ product-definition.md
│   │   ├─ domain-model.md
│   │   ├─ user-flows.md
│   │   ├─ system-context.md
```

```
│   │   ├─ architecture-blueprint.md
│   │   ├─ integration-map.md
│   │   ├─ data-model.md
│   │   ├─ api-contracts.md
│   │   ├─ technical-risk-assessment.md
│   │   └─ implementation-readiness.md
│   │
│   ├─ build-os/
│   │   ├─ project-generator.md
│   │   ├─ project-config-schema.md
│   │   ├─ repository-template.md
│   │   ├─ coding-standards.md
│   │   ├─ code-review-rules.md
│   │   ├─ testing-strategy.md
│   │   ├─ migration-policy.md
│   │   ├─ dependency-policy.md
│   │   ├─ ci-cd-pipeline.md
│   │   └─ deployment-policy.md
│   │
│   └─ feedback-os/
│       ├─ task-reflection.md
│       ├─ learning-database.md
│       ├─ agent-performance.md
│       ├─ improvement-proposals.md
│       ├─ failure-analysis.md
│       ├─ knowledge-library.md
│       ├─ scaling-policy.md
│       └─ governance-model.md
│
├─ agents/
│   ├─ shared/
│   │   ├─ system-prompt.md
│   │   ├─ output-format.md
│   │   ├─ handoff-template.md
│   │   └─ review-template.md
│   │
```

```
│   ├─ product-strategist/
│   │   ├─ agent-config.yaml
│   │   ├─ prompt.md
│   │   ├─ tools.yaml
│   │   └─ workflows.md
│   │
│   ├─ system-architect/
│   ├─ software-engineer/
│   ├─ qa-reviewer/
│   └─ devops-platform/
│
├─ orchestrator/
│   ├─ docker-compose.yml
│   ├─ .env.example
│   ├─ config/
│   │   ├─ models.yaml
│   │   ├─ routing.yaml
│   │   ├─ permissions.yaml
│   │   ├─ repositories.yaml
│   │   └─ workflows.yaml
│   │
│   ├─ scripts/
│   │   ├─ start.sh
│   │   ├─ stop.sh
│   │   ├─ bootstrap.sh
│   │   └─ healthcheck.sh
│   │
│   └─ runtime/
│       ├─ logs/
│       ├─ memory/
│       └─ cache/
│
├─ templates/
│   ├─ nextjs-saas-template/
│   ├─ fullstack-template/
│   ├─ backend-service-template/
```

```
│   └─ ai-feature-template/
│
├─ mcp/
│   ├─ github/
│   │   ├─ config.yaml
│   │   └─ permissions.md
│   ├─ slack/
│   ├─ docs/
│   └─ storage/
│
├─ projects/
│   ├─ starter-projects/
│   └─ generated-projects/
│
├─ knowledge/
│   ├─ patterns/
│   ├─ anti-patterns/
│   ├─ postmortems/
│   ├─ architecture-decisions/
│   └─ reusable-blueprints/
│
├─ audits/
│   ├─ access-review.md
│   ├─ security-audit.md
│   └─ model-usage-audit.md
│
└─ tooling/
    ├─ cli/
    │   ├─ create-project.ts
    │   ├─ register-agent.ts
    │   └─ run-workflow.ts
    └─ utilities/
        ├─ validate-config.ts
        └─ sync-foundation.ts
```

2. What each top-level folder means

`foundation/`

This is the **brain and heart** of the whole system.

This is where your six phases live.

`agents/`

This is where each Chappie becomes real as an operational role.

`orchestrator/`

This is the runtime layer for your OpenClow/OpenCloud-style orchestration.

`templates/`

This is your boilerplate factory.

`mcp/`

This is where your tool connectors live, such as GitHub, Slack, docs, storage, and others.

`projects/`

This is where generated projects or starter projects are tracked.

`knowledge/`

This is the long-term memory of your AI organization.

---

3. Recommended implementation order inside the repo

Do it in this order:

Step 1

Create only these first:

```
foundation/
agents/
orchestrator/
templates/
docs/
```

Step 2

Fill `foundation/` first.

Step 3

Create only 4 initial agents:

- product-strategist
- system-architect
- software-engineer
- qa-reviewer

Step 4

Create one template first, for example:

```
templates/nextjs-saas-template/
```

Step 5

Only then connect the orchestrator.

That way you do not build too much too early.

---

4. Helicopter-view deployment architecture

Below is the big picture.

```
flowchart TD
    A[Human Owner / Engineering Lead] --> B[AI Orchestrato
r on Dedicated Machine]
    B --> C1[Product Strategist Agent]
    B --> C2[System Architect Agent]
    B --> C3[Software Engineer Agent]
    B --> C4[QA Reviewer Agent]
    B --> C5[DevOps Agent]

    B --> D1[GitHub Repository]
    B --> D2[MCP Connectors]
    B --> D3[Model Providers / Cloud LLM APIs]
    B --> D4[Knowledge / Memory Store]
    B --> D5[Logs / Audit / Observability]

    D2 --> E1[GitHub MCP]
    D2 --> E2[Slack MCP]
    D2 --> E3[Docs MCP]
    D2 --> E4[Storage MCP]

    C1 --> F[Project Specs / PRD / Design Artifacts]
    C2 --> F
    C3 --> G[Generated Code / Boilerplate / Features]
    C4 --> H[Review Findings / QA Reports]
    C5 --> I[CI/CD / Deployment Pipeline]

    D1 --> F
    D1 --> G
    G --> I
```

5. Practical machine-by-machine view

Machine A — Your local machine

This is where **you** work.

Use it for:

- authoring foundation files
- reviewing agent output
- editing repo manually
- using Cursor locally if needed
- approving major decisions

You should **not** treat your laptop as the permanent orchestrator host.

Machine B — Dedicated AI machine / VPS / Mac Mini

This is where the **orchestrator lives**.

Use it for:

- running OpenClow/OpenCloud orchestrator
- running agent workflows
- connecting to GitHub
- connecting to model APIs
- storing runtime logs
- calling MCP tools
- running automation pipelines

This is the **AI operations hub**.

---

6. Security boundary you should imagine

Your dedicated AI machine should **not** have unlimited access.

The safe model is:

```
Dedicated AI Machine
  ├ can access selected GitHub repos
  ├ can access selected MCP tools
  ├ can access model APIs
  ├ can access staging pipeline if approved
  └ cannot access your whole personal computer
```

That is the correct mental model.

The AI machine is **a controlled employee workspace**, not a god-machine.

---

7. What to do after installing the orchestrator

This is the part you kept asking about, so here is the exact mental flow.

After installation, do not start by “teaching the AI everything automatically”

Do this instead:

Step 1 — Mount the foundation into the repo

Put your Phase 1–6 files into the repository.

Step 2 — Configure the orchestrator to read the foundation

This means:

- set which folders are authoritative
- set which roles exist
- set which workflows exist
- set which tools each role may use

Step 3 — Register agents

You create the first agents as runtime identities inside the orchestrator.

Example:

```
product-strategist
system-architect
software-engineer
qa-reviewer
```

Step 4 — Assign model/provider routing

Example:

- strategist uses strong reasoning model
- engineer uses code-optimized model
- QA uses review-optimized model

Step 5 — Connect tool permissions

Example:

- product agent can read docs and GitHub issues
- architect can read specs and repo
- engineer can read/write code in allowed repos
- QA can read repo and run tests
- DevOps can read CI config, maybe deploy only to staging

Step 6 — Create first workflow

Example workflow:

```
feature-request
→ strategist
→ architect
→ engineer
```

```
→ QA
→ human approval
```

Step 7 — Run one pilot project

Do not start with production.

Start with:

- one small SaaS module
- one internal tool
- one demo feature

---

8. Do you need to install Cursor, Cloud Code, GitHub MCP, and others?

The clean answer is:

OpenClow/OpenCloud/orchestrator

This is the **manager / workflow engine**.

Model providers / cloud code / LLM access

These are the **brains**.

Cursor

This is a **developer interface**, not the orchestrator itself.

MCP tools

These are the **hands and eyes** for accessing GitHub, docs, Slack, storage, etc.

So the picture is:

```
Orchestrator = manager
LLM / cloud code = thinking engine
```

```
MCP = tool access layer
GitHub = codebase and source of truth
Cursor = human collaboration interface
```

So yes, you usually configure these separately.

They do not magically become one thing by themselves.

---

9. Recommended first real setup

Here is the simplest real setup I recommend for version 1.

Dedicated machine

- Ubuntu VPS or mini PC
- Docker installed
- Git installed
- Node and Python installed

Orchestrator

- your OpenClow/OpenCloud-style orchestrator
- one runtime project folder
- logs and config mounted as volumes

Model layer

- OpenAI / Claude / other APIs
- maybe one coding-specialized model and one reasoning model

Tool layer

- GitHub MCP
- docs MCP

- storage MCP
- maybe Slack later, not first

Repo layer

- one main `ai-software-factory` repo
- one generated test project repo

Human interface

- GitHub PRs
- Cursor locally
- manual approval checkpoints

That is enough to make version 1 real.

---

10. What your first pilot should look like

Do not let the first pilot be “build my entire company platform.”

Use something like this:

Pilot idea

“Generate a small internal SaaS starter app”

Scope:

- auth
- dashboard
- one CRUD module
- tests
- CI pipeline

Workflow:

1. human gives request

2. strategist creates scope
3. architect creates design
4. engineer scaffolds code
5. QA reviews
6. human approves
7. optional DevOps prepares staging

That will give you the clearest picture.

---

11. What success looks like after implementation

You know this is working when:

- the orchestrator can trigger a full workflow
- each agent has a clear role
- the repo foundation is actually used
- one feature can move from request → design → code → review
- the system does not require manual rewriting at every step
- the feedback loop produces improvements

---

12. The clearest mental image

The easiest way to imagine the whole thing is this:

The repository

is the **school + constitution + company handbook**

The orchestrator

is the **operations manager**

The agents

are the **employees / children you trained**

The model providers

are the **brains / cognitive engines**

MCP and integrations

are the **tools and senses**

GitHub and project repos

are the **workspace**

You

are the **founder, parent, and final decision-maker**

That is the cleanest helicopter view.

---

13. My recommended next three moves

First, create the repository skeleton exactly like above.

Second, write only these files first:

- `foundation/human-intent-os/mission.md`
- `foundation/agent-foundation-os/task-lifecycle.md`
- `foundation/role-definition-os/role-map.md`
- `orchestrator/config/workflows.yaml`
- `agents/software-engineer/prompt.md`

Third, design your first pilot workflow:

- feature request
- architecture draft

- code generation
- QA review

That is the smallest path to reality.

▼ Claude Guideline

```
# Claude Code Setup Blueprint for a Human-Centric Newborn
AI Team

## Purpose

This document defines the most complete practical setup fo
r building the project with Claude Code while ensuring one
non-negotiable rule:

**the system must never proceed into autonomous project de
velopment unless it first behaves like a newborn baby unde
r the Human Intent OS.**

That means:

* it starts small
* it asks before acting
* it learns from human guidance
* it does not assume maturity
* it does not bypass foundation rules
* it does not self-promote into autonomy without passing g
ates

---

# 1. Core operating principle

## 1.1 Human-centric newborn mode is the default
```

```
All Claude Code workflows begin in **Newborn Mode**.

Newborn Mode means:

* Claude is not treated as a senior engineer
* Claude is not allowed to define product direction alone
* Claude is not allowed to silently change architecture
* Claude is not allowed to edit protected files unless exp
licitly permitted
* Claude must explain plans first
* Claude must ask for approval at key checkpoints
* Claude must reference the foundation before generating d
eliverables

## 1.2 Maturity is earned, not assumed

The team can only move from:

* Newborn
* Guided Child
* Supervised Junior
* Trusted Specialist

after explicit human approval and evidence from reflectio
n, review, and quality gates.

## 1.3 The foundation is law

Claude Code must always comply with:

* `foundation/human-intent-os/`
* `foundation/agent-foundation-os/`
* `foundation/role-definition-os/`
* `foundation/design-os/`
* `foundation/build-os/`
```

```
* `foundation/feedback-os/`

If a requested action conflicts with the foundation, Claud
e must stop and escalate.

---

# 2. What Claude Code is in this system

Claude Code is the main runtime engine for your AI enginee
ring organization.

In this setup, Claude Code is used as:

* the day-to-day execution brain
* the interface for engineering work
* the role-switching engine for specialist agents
* the foundation-aware project worker
* the review and reflection partner

Claude Code is **not** the owner, not the policy-maker, an
d not the final approver.

The human remains:

* founder
* parent
* engineering director
* policy approver
* release gatekeeper

---

# 3. End-to-end business flow

## Stage A — Build the foundation
```

```
Create and maintain the six core foundation layers:

* Human Intent OS
* Agent Foundation OS
* Role Definition OS
* Design OS
* Build OS
* Feedback OS

These define how Claude should think, behave, collaborate,
design, implement, and learn.

## Stage B — Install Claude Code environment

Claude Code is installed in the engineering environment an
d attached to the repository.

## Stage C — Load the foundation into Claude’s working con
text

Claude must read the repository guidance and rules before
any build workflow starts.

## Stage D — Create a newborn team

Create specialist subagents and skills, but force them all
to begin in Newborn Mode.

## Stage E — Run guided workflows only

Start with tightly supervised workflows such as:

* define product
* design system
* scaffold repo
```

```
* generate one feature
* review code
* reflect

## Stage F — Learn and improve

Every task produces reflection, findings, and proposed upd
ates.

## Stage G — Scale only after trust is proven

Expand permissions, tools, and autonomy only after repeate
d successful cycles.

---

# 4. Required repository structure

```text
ai-software-factory/
├─ CLAUDE.md
├─ MEMORY.md
├─ README.md
├─ docs/
├─ foundation/
│  ├─ human-intent-os/
│  ├─ agent-foundation-os/
│  ├─ role-definition-os/
│  ├─ design-os/
│  ├─ build-os/
│  └─ feedback-os/
├─ .claude/
│  ├─ settings.json
│  ├─ agents/
│  ├─ skills/
│  ├─ hooks/
```
```

```
│   └─ plugins/
├─ workflows/
├─ templates/
├─ knowledge/
├─ projects/
└─ orchestrator/
```

---

5. Mandatory root files

5.1 `CLAUDE.md`

This is the main project instruction file.

It should contain:

* project purpose
* non-negotiable foundation rules
* newborn behavior requirements
* approval gates
* architecture constraints
* coding expectations
* escalation rules
* where Claude must look first

Suggested top sections:

* Mission
* Newborn Team Rule
* Foundation Loading Order
* Prohibited Actions
* Required Workflow Order
* Role Boundaries
* Protected Files

```
* Approval Gates

## 5.2 `MEMORY.md`

This is the short persistent project memory.

Keep it concise.
Store only:

* stable project facts
* current repo conventions
* important environment notes
* repeated pitfalls

Do not put long design docs here.
Use separate files under `knowledge/` for detail.

---

# 6. Foundation loading order

Claude must load context in this order:

1. `CLAUDE.md`
2. `foundation/human-intent-os/`
3. `foundation/agent-foundation-os/`
4. relevant `foundation/role-definition-os/<role>/`
5. relevant `foundation/design-os/`
6. relevant `foundation/build-os/`
7. relevant `foundation/feedback-os/`
8. current project docs
9. current code context
10. user request

If required context is missing, Claude must not improvise
high-risk decisions.
```

```
---

# 7. Newborn Mode contract

Every agent and workflow must begin with the following beh
avioral contract.

## 7.1 Newborn rules

Claude must:

* begin in plan mode
* explain what it understands first
* list assumptions explicitly
* ask before making irreversible changes
* avoid touching protected files automatically
* avoid broad refactors unless requested
* follow the smallest safe next step
* propose, not overrule
* defer to foundation if conflict appears

Claude must not:

* silently invent requirements
* silently alter architecture
* self-upgrade permissions
* bypass human approval gates
* run dangerous commands without policy approval
* act like a mature autonomous organization before proving
reliability

## 7.2 Newborn checkpoint before every workflow

Before doing substantive work, Claude must answer:
```

```
* What is my role for this task?
* What foundation files apply?
* What output is expected?
* What should I not do?
* What assumptions am I making?
* What approval do I need before proceeding?

If this checkpoint is skipped, the workflow is invalid.

---

# 8. Claude Code features to use at maximum potential

## 8.1 Skills

Use `.claude/skills/` to package repeatable workflows.

Recommended skills:

* `newborn-gate`
* `define-product`
* `design-system`
* `generate-architecture`
* `generate-feature`
* `review-code`
* `reflect-task`
* `foundation-audit`
* `guarded-refactor`
* `release-readiness`

Each skill should contain:

* purpose
* invocation guidance
* required inputs
* required context files
```

```
* exact output format
* refusal rules when foundation is missing

## 8.2 Subagents

Use `.claude/agents/` to create role-based subagents.

Starter agents:

* Product Strategist
* System Architect
* Software Engineer
* QA Reviewer
* DevOps Platform

Every subagent must inherit the newborn contract.

## 8.3 Hooks

Use hooks to enforce deterministic behavior.

Required hooks:

* block edits to protected files
* block execution when foundation files are missing
* auto-run lint/format after edits
* auto-run newborn gate before dangerous actions
* log reflection after task completion
* audit changes to CLAUDE.md, foundation, CI, infra, auth,
and migrations

## 8.4 MCP

Use MCP only for controlled access.

Recommended MCP order:
```

```
1. GitHub
2. Docs/Notion/internal docs
3. Storage
4. Slack later

Do not start with too many MCP tools.

## 8.5 Permission modes

Default mode should be the most restrictive practical mod
e.

Recommended progression:

* Plan mode by default
* Normal mode for approved low-risk work
* Auto-accept only for highly trusted low-risk automation

---

# 9. Complete `.claude/` setup

```text
.claude/
├─ settings.json
├─ agents/
│  ├─ product-strategist.md
│  ├─ system-architect.md
│  ├─ software-engineer.md
│  ├─ qa-reviewer.md
│  └─ devops-platform.md
├─ skills/
│  ├─ newborn-gate/
│  │  └─ SKILL.md
│  ├─ define-product/
```
```

```
│   │   └─ SKILL.md
│   ├─ design-system/
│   │   └─ SKILL.md
│   ├─ generate-architecture/
│   │   └─ SKILL.md
│   ├─ generate-feature/
│   │   └─ SKILL.md
│   ├─ review-code/
│   │   └─ SKILL.md
│   ├─ reflect-task/
│   │   └─ SKILL.md
│   └─ foundation-audit/
│       └─ SKILL.md
├─ hooks/
│   ├─ verify-foundation.sh
│   ├─ protect-critical-files.sh
│   ├─ run-quality-gates.sh
│   └─ write-reflection.sh
└─ plugins/
```

---

10. Required skills

10.1 `newborn-gate`

This is the most important skill.

Purpose:

* stop all workflows unless the task is aligned with Human Intent OS
* force plan-first behavior
* force assumption declaration
* refuse unsafe autonomy

```
It should check:

* required foundation files exist
* current role is declared
* output format is known
* protected files list is loaded
* required approvals are identified

## 10.2 `define-product`

Role: Product Strategist

Output:

* problem statement
* user stories
* constraints
* acceptance criteria
* unresolved questions

## 10.3 `design-system`

Role: System Architect

Output:

* system context
* architecture blueprint
* domain model
* integration map
* technical risks

## 10.4 `generate-feature`

Role: Software Engineer
```

```
Output:

* implementation plan
* files to change
* code changes
* tests
* documentation updates
* risk notes

## 10.5 `review-code`

Role: QA Reviewer

Output:

* pass/fail
* critical issues
* major issues
* minor improvements
* regression risks

## 10.6 `reflect-task`

Role: reviewer/self-reflection

Output:

* expected outcome
* actual outcome
* what failed
* what was missing
* what should be improved in foundation or skill packs

---
```

```
# 11. Required subagents

## 11.1 Product Strategist

Owns:

* intent shaping
* product definition
* user stories
* acceptance criteria

Cannot:

* finalize architecture
* implement code
* invent technical decisions as facts

## 11.2 System Architect

Owns:

* architecture design
* system boundaries
* data flow
* service decomposition

Cannot:

* redefine business intent
* silently expand scope

## 11.3 Software Engineer

Owns:

* code implementation
```

```
* tests
* implementation notes

Cannot:

* rewrite architecture without escalation
* change product scope silently

## 11.4 QA Reviewer

Owns:

* quality verification
* requirements checking
* defect finding
* regression awareness

Cannot:

* silently redefine requirements

## 11.5 DevOps Platform

Owns:

* CI/CD
* deployment readiness
* environment setup
* runtime policy

Cannot:

* bypass governance for production changes

---
```

```
# 12. Dedicated machine setup

## 12.1 Purpose of the dedicated machine

This machine is the controlled workspace for Claude-enable
d engineering workflows.

It is not your personal machine.
It is not a god-box.
It is a controlled worker environment.

## 12.2 Recommended machine profile

For v1:

* Ubuntu VPS or dedicated Linux machine
* Docker installed
* Git installed
* Node installed
* Python installed
* restricted SSH access
* disk encryption if possible
* separate non-root working user

## 12.3 What runs there

* Claude Code CLI/runtime environment
* repo checkout
* MCP client configuration
* hooks
* controlled scripts
* optional orchestrator wrapper
* logs and audit artifacts

---
```

```
# 13. Security boundaries for the dedicated machine

The machine should:

* access only selected repositories
* use least-privilege tokens
* have separate project workspace paths
* not mount your whole personal filesystem
* not have unrestricted production access
* prefer staging-only deployment rights first

Recommended access pattern:

* one machine identity for the automation environment
* scoped GitHub tokens or app installation
* scoped MCP connectors
* separate secrets for staging and production
* protected human approval for dangerous workflows

---

# 14. What to do immediately after Claude Code is installe
d

## Step 1 — Clone the repo

Clone the AI software factory repository onto the dedicate
d machine.

## Step 2 — Create root guidance

Add:

* `CLAUDE.md`
* `MEMORY.md`
* foundation folders
```

```
* `.claude/` structure

## Step 3 — Configure settings

Set the default permission mode conservatively.
Configure hooks.
Register MCP tools.
Restrict allowed commands where appropriate.

## Step 4 — Create the newborn gate skill

This must exist before any other autonomous workflow.

## Step 5 — Create the 4 core subagents

Start with:

* Product Strategist
* System Architect
* Software Engineer
* QA Reviewer

## Step 6 — Add protected files policy

Protect:

* `CLAUDE.md`
* `foundation/**`
* secrets/config
* infra/CI
* database migrations
* auth and security controls

## Step 7 — Run foundation audit

Claude must verify that the repo is structurally ready.
```

```
## Step 8 — Run one pilot workflow only

Start with:

* one small feature
* one scaffold task
* one architecture draft

Do not start with full autonomy.

---

# 15. Recommended workflow sequence

Every workflow should follow this order:

1. newborn gate
2. role selection
3. context loading
4. task plan
5. human approval if required
6. execution
7. review
8. reflection
9. memory update if needed

If any step is skipped, the workflow is incomplete.

---

# 16. First pilot workflow

## Objective

Build one small internal SaaS starter feature safely.
```

```
## Example

Feature: visitor approval workflow

## Sequence

1. `newborn-gate`
2. Product Strategist produces product-definition
3. System Architect produces architecture blueprint
4. Software Engineer produces implementation plan first
5. Human approves plan
6. Software Engineer writes code and tests
7. QA Reviewer reviews
8. Reflection is written
9. Foundation updates are proposed only, not auto-applied

---

# 17. Hooks policy

## Hook 1 — Verify foundation before write actions

If foundation files are missing or invalid, block executio
n.

## Hook 2 — Protect critical files

If Claude tries to modify protected files, block or requir
e special approval.

## Hook 3 — Run quality gates after edits

Run formatter, linter, and relevant tests.

## Hook 4 — Force reflection after significant tasks
```

```
Write reflection output to feedback records.

## Hook 5 — Audit sensitive changes

Sensitive changes must produce an audit note.

---

# 18. Maturity ladder

## Level 0 — Newborn

* plan only
* no autonomous code changes without approval
* no dangerous tools
* no broad write permissions

## Level 1 — Guided Child

* small approved code changes
* can run low-risk local commands
* still asks before substantial edits

## Level 2 — Supervised Junior

* can execute scoped feature tasks
* can write tests and docs automatically
* still requires review gates

## Level 3 — Trusted Specialist

* can handle a full role workflow in bounded scope
* can participate in larger pipelines
* still cannot bypass governance
```

```
The default level is always Level 0 until promoted.

---

# 19. What success looks like

The setup is working when:

* Claude always references the foundation
* workflows stop when foundation is missing
* the newborn gate runs before major work
* every agent behaves within role boundaries
* code generation follows architecture
* review and reflection happen every cycle
* autonomy increases only after evidence, not optimism

---

# 20. Final principle

Do not try to create an adult AI team on day one.

Create a **human-centric newborn team** first.
That means:

* small scope
* guided learning
* deterministic guardrails
* explicit approvals
* reflection after every task
* maturity earned through repeated trustworthy behavior

That is how Claude Code becomes not just a coding tool, bu
t the engine of a disciplined AI engineering organization.
```

Claude Code supports project guidance through `CLAUDE.md`, has a `/memory` flow for inspecting loaded project rules and memories, supports reusable **skills** via `SKILL.md`, supports **subagents** you can create with `/agents`, and supports **hooks** for deterministic enforcement like blocking edits, running checks, and injecting context. That combination is exactly what makes your "newborn baby first, autonomy later" model practical rather than just conceptual. (Claude API Docs)

The most important operational choice in the document is this: your team should start in **Plan mode / highly restricted mode**, not auto-accept mode. Claude Code's IDE flow supports permission modes, and its settings can be shared through `~/.claude/settings.json`, which is also where hooks, permissions, and MCP servers are configured. (Claude API Docs)

I also aligned the setup around official Claude Code primitives rather than invented abstractions:

- **skills** for reusable workflows and commands, including direct invocation with `/skill-name` (Claude API Docs)
- **subagents** for role-based "Chappies" like strategist, architect, engineer, and QA reviewer (Claude API Docs)
- **hooks** for hard guardrails, because hooks are deterministic and better for enforcing rules than hoping the model remembers every time (Claude API Docs)
- **MCP** for controlled tool access to GitHub, docs, storage, and later Slack (Claude API Docs)
- optional **Claude Agent SDK** later if you want to turn this from repo-driven Claude Code usage into programmable production agents with the same tool loop and context model (Claude API Docs)

The document's central rule is intentionally strict:

No project workflow should proceed unless the newborn gate passes first.

That means:

- foundation files must exist
- the active role must be declared
- protected files must be guarded

- assumptions must be explicit
- approval gates must be identified before code generation begins

That is the safest way to make Claude Code comply with your Human Intent OS instead of behaving like a prematurely autonomous senior engineer.

The next highest-value artifact is a concrete starter pack with these exact files:

- `CLAUDE.md`
- `.claude/settings.json`
- `.claude/agents/product-strategist.md`
- `.claude/agents/system-architect.md`
- `.claude/agents/software-engineer.md`
- `.claude/agents/qa-reviewer.md`
- `.claude/skills/newborn-gate/SKILL.md`
- `.claude/skills/generate-feature/SKILL.md`
- `.claude/hooks/protect-critical-files.sh`

I can generate that starter pack next.